

CS 490: R&D Project - Final Report

Patrol Mission with Safety Constraints

March - April 2026

Contents

1	Motivation	1
2	Patrol Mission: Code Overview	1
2.1	Automaton	1
2.1.1	Explanation of the Functions	1
2.1.2	Idea	1
2.1.3	Dynamic Frontier Mechanism	1
2.1.4	Reward Structure	2
2.1.5	How to Model a Different Automaton	2
2.2	Environment (MDP)	2
2.2.1	Core Logic	2
2.3	Component Breakdown	2
2.3.1	How to Model a Different Environment	3
2.4	Q-Learning Algorithm	3
2.4.1	State and Action Space	3
2.4.2	The Bellman Equation	3
2.4.3	Exploration vs. Exploitation	3
2.5	System Implementation and Evaluation	3
2.5.1	Training Phase	4
2.5.2	Convergence Analysis	4
2.5.3	Policy Evaluation and Visualization	4
3	Future Work	4
4	References	4

1 Motivation

The development of this framework is driven by the limitations observed in existing Logic-Constrained Reinforcement Learning (LCRL) which often:

- Require predefined, rigid automaton files
- Utilize fixed accepting frontier logic, which limits the complexity of the tasks the agent can learn
- Assign rewards automatically, making it difficult for developers to customize priorities or handle dynamic task requirements

Our goal is to build a flexible framework where the automaton logic is directly programmable, allowing for richer temporal tasks and manual control over frontier updates.

2 Patrol Mission: Code Overview

2.1 Automaton

2.1.1 Explanation of the Functions

1. `__init__(self):`

- Idea: Initializes the mission’s starting point
- Key Variables:
 - `accepting_labels`: The full list of goals (A, B, C).
 - `frontier`: The current “to-do list.” As the agent visits targets, they are removed from here.
 - `needs_base_reset`: A state variable (a “flag”) that remembers if the agent made a mistake and needs to return to the Base.

2. `transition(self, label, verbose=False):` Every time the agent moves to a new tile, the environment tells the automaton the label of that tile.

- Safety Logic: It checks if the label is `restricted` (Game Over) or `risk` (Compromised).
- Mission Logic: It checks if the label is in the `frontier`. If it is, it removes the label from the frontier function, so that the agent goes to a different checkpoint, as required.
- Reset Logic: Once the `frontier` is empty (all targets visited), it refills the list, creating an infinite loop (the “Büchi” part).

3. `get_current_state(self):`

- Idea: Provides the “Logical State” to the Q-Learning agent.
- A standard RL agent only knows its position. This function tells where it is and which checkpoints are yet to be found.

2.1.2 Idea

This file represents the Automaton part of the Product MDP.

Normally, an RL agent has no memory. By combining the Physical State (Grid) with the Logical State (Automaton), we create a “Product State.”

2.1.3 Dynamic Frontier Mechanism

Unlike fixed LCRL frontiers, we define frontier updates manually to allow for complex sequencing. The frontier tracks the “remaining” targets for the current cycle. This mechanism can be programmed to require any-order visits (A, B, C) or strict sequences (e.g., A then B then C).

2.1.4 Reward Structure

The reward signal is the primary driver of behavior.

- **Progress Rewards:** +50 for visiting a target in the current frontier
- **Cycle Rewards:** +150 for completing a full Büchi cycle
- **Penalty:** -100 for restricted zone hits and -5 for entering risk zones
- **Efficiency:** A small negative reward (-0.1) is applied to every safe step to encourage the agent to find the shortest path between targets

2.1.5 How to Model a Different Automaton

If a user wants to change the mission (e.g., “Visit A then B, but never touch C”), they need to modify a few specific areas:

- **Changing the goals:** To change the targets, update the `accepting_labels`.
- **Changing the Sequence Logic:** Currently, the code allows the agent to visit A, B, and C in any order. If the user wants a strict sequence, the frontier logic would have to be changed to have the specific checkpoint that is to be visited.
- **Changing the Safety Constraints:** The code currently avoids the `restricted` area. If the user wants to have a different safety constraint, they will have to associate a huge negative reward with it and end the episode there.
- **Changing the Rewards:** The rewards control the agent’s behaviour. In the current code, the constraints are of two types:
 - Hard Constraints: Use large negative rewards (-100) and set `is_terminal = True`.
 - Soft Constraints: Use small negative rewards (-5) and set `is_terminal = False` to encourage the agent to stay away without ending the game.

2.2 Environment (MDP)

2.2.1 Core Logic

The most important part of this file is the Labeling Function. In standard RL, the agent only sees coordinates (position). The environment must translate them into labels, so that the automaton can understand.

The `get_label(self, position)` maps every (x, y) coordinate in the 10×10 grid to a semantic label.

2.3 Component Breakdown

1. **The Grid (10×10):** The physical world is a discrete state space. The boundaries are strictly enforced in `main.py` using `max(0, min(9, ...))`, ensuring the agent never leaves this environment.
2. **Targets (A, B, C):** The coordinates (1, 2), (8, 2), and (5, 8) are the only places where the agent can receive a “Progress” reward (+50) from the Automaton.
3. **Restricted Zones (The “Hard” Constraints):** If `get_label` returns “restricted,” the Automaton triggers `done = True`, thus ending the episode.
4. **Risk Zones (The “Soft” Constraints):** Unlike restricted zones, these do not end the episode. Instead, they change the agent’s internal “logical state” to Compromised. The agent can still move but is penalized and needs to perform a specific action (visiting the Base) to recover.

2.3.1 How to Model a Different Environment

If a user wants to model a different physical scenario, they would have to modify these elements:

- **Grid Layout:** Change the `self.grid_size` (from 10, currently).
- **Target Locations:** Change `self.targets`.
- **Map Layout:** Change the coordinates in `self.restricted` to create different wall structures

The user can also add other items to model an environment, as required.

2.4 Q-Learning Algorithm

The agent utilizes **Q-Learning**, a model-free temporal difference (TD) learning algorithm. The goal is to learn an optimal policy π^* that guides the agent to complete infinite Büchi cycles while adhering to safety constraints.

2.4.1 State and Action Space

- **Product State Space (S):** To handle the Büchi condition, we use a Product MDP state:

$$s = (pos, automaton_state)$$

where $pos \in \{0 \dots 9\}^2$ and *automaton_state* tracks the unvisited targets (frontier) and the safety flag. This ensures the agent “remembers” its progress.

- **Action Space (A):** The agent can move in four cardinal directions: {UP, DOWN, LEFT, RIGHT}.

2.4.2 The Bellman Equation

The core of the learning process is the Q-table update, which adjusts the estimated value of an action based on the reward received (r) and the discounted potential of the next state:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Where:

- α (Learning Rate): Set to 0.2, determining how much new information overrides old estimates.
- γ (Discount Factor): Set to 0.95, emphasizing long-term rewards (crucial for multi-step patrol missions).

2.4.3 Exploration vs. Exploitation

We implement an **ϵ -greedy strategy** to solve the exploration-exploitation trade-off:

- **Exploration:** With probability ϵ , the agent chooses a random action to discover the grid and automaton transitions.
- **Exploitation:** With probability $1 - \epsilon$, the agent selects the action with the highest Q -value.
- **Decay:** ϵ starts at 1.0 and decays by a factor of 0.998 each episode, eventually reaching a minimum of 0.05. This ensures the agent stabilizes as it gains experience.

2.5 System Implementation and Evaluation

The execution flow (managed in `main.py`) integrates the environment, the task logic, and the learning agent into a unified pipeline.

2.5.1 Training Phase

The agent is trained over 5,000 episodes. Each episode begins at a fixed starting position (5, 5) with a reset automaton. An episode terminates if:

1. The agent enters a **restricted** zone.
2. The agent violates protocol by visiting a target while in a **compromised** state.
3. The maximum step limit (500) is reached.

2.5.2 Convergence Analysis

To verify learning, we track the total reward per episode.

- **Variance:** High initial variance is expected due to ϵ -greedy exploration and “accidental” deaths in restricted zones.
- **Stability:** A rising rolling average that plateaus indicates the agent has found an efficient path to complete Büchi cycles while avoiding penalties.

2.5.3 Policy Evaluation and Visualization

After training, the ϵ parameter is effectively set to 0 to evaluate the learned **Greedy Policy**.

- **Path Visualization:** The evaluation path is plotted on a 10×10 grid. A successful agent will show a repeating path through A, B, and C. If the agent hits a **risk** zone, the path should deviate toward the **base** before returning to the targets.
- **Cycle Count:** We measure the number of full Büchi cycles (visiting A, B, and C in any order) completed within an 800-step window. High cycle counts with zero “terminal” failures confirm the robustness of the learned safety-aware patrol mission.

3 Future Work

While the current framework successfully respects safety constraints and completes patrol cycles, several avenues for expansion exist:

- **Comparison with LCRL:** Benchmarking the programmable framework against existing Logic-Constrained Reinforcement Learning tools to quantify flexibility gains
- **Complex Temporal Missions:** Expanding to multi-stage sequences (e.g., $A \rightarrow B \rightarrow C$ then $D \rightarrow E$) and dynamic triggers
- **Continuous Environments:** Moving beyond the 10×10 grid to high-fidelity physics simulators and non-discrete coordinate systems
- **LTL-to-Code Automation:** Developing a compiler to automatically generate automaton logic from formal Linear Temporal Logic (LTL) specifications

4 References

References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [2] H. Hasanbeig, D. Kroening, and A. Abate, “Certified reinforcement learning with logic guidance,” *Artificial Intelligence*, vol. 322, p. 103949, 2023.
- [3] E. M. Hahn et al., “Omega-Regular Decision Processes,” *arXiv preprint arXiv:2312.08602*, 2023.